is, `save!`and `update!`), which are stricter in that they raise the exception `ActiveRecord::RecordInvalid` if validation fails. A quick example to illustrate:

```
class User < ActiveRecord::Base
  validates :name, presence: true
end

user = User.new
user.save  # => false
user.save! # => ActiveRecord::RecordInvalid: Validation failed: Name
can't be blank
```

You can learn more about validations in the Active Record Validations guide.

## 7 Callbacks

Active Record callbacks allow you to attach code to certain events in the life-cycle of your models. This enables you to add behavior to your models by transparently executing code when those events occur, like when you create a new record, update it, destroy it and so on. You can learn more about callbacks in the Active Record Callbacks guide.

## 8 Migrations

Rails provides a domain-specific language for managing a database schema called migrations. Migrations are stored in files which are executed against any database that Active Record supports using `rake`.

Here's a migration that creates a table:

```
class CreatePublications < ActiveRecord::Migration
  def change
    create_table :publications do |t|
      t.string :title
      t.text :description
      t.references :publication_type
      t.integer :publisher_id
      t.string :publisher_type
      t.boolean :single_issue

      t.timestamps null: false
    end
    add_index :publications, :publication_type_id
  end
end
```

Rails keeps track of which files have been committed to the database and provides rollback features. To actually create the table, you'd run `rake db:migrate` and to roll it back, `rake db:rollback`.

Note that the above code is database-agnostic: it will run in MySQL, PostgreSQL, Oracle and others. You can learn more about migrations in the Active Record Migrations guide.